UNIVERSIDADE FEDERAL DO CEARÁ

TECHNICAL REPORT

Aluno: Willian Mota e Ruan De Lima

1. Introdução

Na avaliação foram utilizados dois datasets fornecidos pelo professor da plataforma kaggle no repositório da turma no github, os datasets foram um de classificação e o outro de regressão

Classificação: **Chocolate Bar Ratings**

O dataset de classificação utilizado foi o Chocolate Bar Ratings, obtido na plataforma kaggle, ele contém informações sobre os tipos de chocolates que são avaliados. O objetivo é classificar os sabores dos chocolates numa escala de 1 a 5, sendo 1 o pior e 5 o melhor.

Regressão: **Expectativa de vida (OMS)** O dataset para regressão também extraído do kaggle, contém informações sobre Análise estatística sobre fatores que influenciam a expectativa de vida.

2. Resultados e discussão

Nesta seção deve-se descrever como foram as resoluções de cada questão. Crie sessões indicando a questão e discuta a implementação e resultados obtidos nesta. Explique o fluxograma do processo de cada questão, indicando quais processamentos são realizados nos dados. Sempre que possível, faça gráficos, mostre imagens, diagramas de blocos para que sua solução seja a mais completa possível. Discuta sempre sobre os números obtidos em busca de motivos de erros e acerto.

Questão 1 **Objetivo do Código**

3. Limpar e preparar os dados de um arquivo CSV (remoção de valores ausentes e colunas desnecessárias).
4. Converter a coluna "Cocoa\nPercent" de texto percentual para fração decimal.
5. Salvar o DataFrame processado em um novo arquivo para uso posterior.

```python
import pandas as pd

df = pd.read_csv("flavors_of_cacao.csv")
print("Resumo:")
print(df.info())

df = df.dropna()

df = df.drop( labels: ["REF", "Review\nDate"], axis=1)

print(df.dtypes)

def percentfloat(df):  1 usage  new *
    return df.apply(lambda x: float(x.strip('%'))/100)

df['Cocoa\nPercent'] = percentfloat(df["Cocoa\nPercent"])

print(df.dtypes)

df.to_csv( path_or_buf: "sabores_de_cacau_ajustado.csv", index=False)
```

Início (Importação da biblioteca)

Leitura do arquivo CSV

Exibição do resumo do DataFrame

Remoção de valores ausentes

Remoção de colunas indesejadas

Exibição dos tipos de dados

Conversão da coluna de percentual

Exibição dos tipos de dados atualizados

Exportação dos dados ajustados

```
Resumo:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1795 entries, 0 to 1794
Data columns (total 9 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   Company
(Maker-if known)         1795 non-null   object
 1   Specific Bean Origin
or Bar Name  1795 non-null   object
 2   REF                               1795 non-null   int64
 3   Review
Date                       1795 non-null   int64
 4   Cocoa
Percent                     1795 non-null   object
 5   Company
Location                  1795 non-null   object
 6   Rating                            1795 non-null   float64
 7   Bean
Type                         1794 non-null   object
 8   Broad Bean
Origin                 1794 non-null   object
dtypes: float64(1), int64(2), object(6)
memory usage: 126.3+ KB
None
Company \n(Maker-if known)            object
Specific Bean Origin\nor Bar Name     object
Cocoa\nPercent                        object
Company\nLocation                     object
Rating                               float64
Bean\nType                            object
Broad Bean\nOrigin                    object
dtype: object
Company \n(Maker-if known)            object
Specific Bean Origin\nor Bar Name     object
Cocoa\nPercent                       float64
Company\nLocation                     object
Rating                               float64
Bean\nType                            object
Broad Bean\nOrigin                    object
dtype: object

Process finished with exit code 0
```

Questão 5

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

file_path = 'Life Expectancy Data.csv'
life_expectancy_data = pd.read_csv(file_path)

life_expectancy_data_cleaned = life_expectancy_data.dropna()

target = 'Life expectancy '
significant_features = ['Schooling', 'Income composition of resources', ' BMI ']

X = life_expectancy_data_cleaned[significant_features]
y = life_expectancy_data_cleaned[target]

X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.7)
plt.plot( *args: [y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r', lw=2)
plt.xlabel('Actual Life Expectancy')
plt.ylabel('Predicted Life Expectancy')
plt.title('Actual vs Predicted Life Expectancy')
plt.show()
```

Importação de Bibliotecas

Leitura e Limpeza dos Dados

Definição do Alvo e das Variáveis

Divisão em Treinamento e Teste

Criação e Treinamento do Modelo

Previsão e Avaliação

Questão 6

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
```

```python
file_path = 'Life Expectancy Data.csv'
life_expectancy_data = pd.read_csv(file_path)

life_expectancy_data_cleaned = life_expectancy_data.dropna()

target = 'Life expectancy '
most_significant_feature = 'Schooling'

X = life_expectancy_data_cleaned[[most_significant_feature]]
y = life_expectancy_data_cleaned[target]

X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

rss = np.sum((y_test - y_pred) ** 2)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f'Residual Sum of Squares (RSS): {rss}')
print(f'Mean Squared Error (MSE): {mse}')
print(f'Root Mean Squared Error (RMSE): {rmse}')
print(f'R-squared (R^2): {r2}')

plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_test[most_significant_feature], y=y_test, alpha=0.7, label='Actual')
plt.plot( *args: X_test[most_significant_feature], y_pred, color='red', label='Regression Line')
plt.xlabel(most_significant_feature)
plt.ylabel('Life Expectancy')
plt.title('Regression Line with Actual Data')
plt.legend()
plt.show()
```

Importação de Bibliotecas

Leitura e Limpeza dos Dados

Definição do Alvo e da Variável Explicativa

Divisão dos Dados

Criação e Treinamento do Modelo

Previsões nos Dados de Teste

Cálculo de Métricas

Exibição dos Resultados

Questão 7

```
import numpy as np
import pandas as pd
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

def k_fold_cross_validation(data, target, model, k=5):  1usage  new*
    indices = np.arange(len(data))
    np.random.shuffle(indices)

    fold_sizes = len(data) // k
    folds = [indices[i * fold_sizes:(i + 1) * fold_sizes] for i in range(k)]

    errors = []

    for i in range(k):
        val_indices = folds[i]
        train_indices = np.setdiff1d(indices, val_indices)

        X_train, X_val = data[train_indices], data[val_indices]
        y_train, y_val = target[train_indices], target[val_indices]

        model.fit(X_train, y_train)

        y_pred = model.predict(X_val)

        error = np.sqrt(mean_squared_error(y_val, y_pred))
        errors.append(error)

    return errors

def calculate_metrics(y_true, y_pred):  1usage  new*
    rss = np.sum((y_true - y_pred) ** 2)
    mse = np.mean((y_true - y_pred) ** 2)
    rmse = np.sqrt(mse)
    r2 = 1 - (rss / np.sum((y_true - np.mean(y_true)) ** 2))
    return {
        'RSS': rss,
```

```python
        'MSE': mse,
        'RMSE': rmse,
        'R^2': r2
    }

file_path = 'Life Expectancy Data.csv'
life_expectancy_data = pd.read_csv(file_path)

data_cleaned = life_expectancy_data.dropna()
data = data_cleaned[['GDP']].values
target = data_cleaned['Life expectancy '].values

data_train, data_test, target_train, target_test = train_test_split( *arrays: data, target, test_size=0.2, random_state=42)

models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=0.1)
}

for name, model in models.items():
    errors = k_fold_cross_validation(data_train, target_train, model, k=5)
    model.fit(data_train, target_train)
    test_pred = model.predict(data_test)
    test_error = np.sqrt(mean_squared_error(target_test, test_pred))

    metrics = calculate_metrics(target_test, test_pred)

    print(f"{name}")
    print(f"Erros por fold: {errors}")
    print(f"Erro médio (validação): {np.mean(errors)}")
    print(f"Erro no conjunto de teste: {test_error}")
    print(f"Métricas: {metrics}\n")

model = LinearRegression()
model.fit(data_train, target_train)
plt.scatter(data, target, color='blue', label='Dados reais')
plt.plot( *args: data, model.predict(data), color='red', label='Ajuste Linear')
```

```python
plt.xlabel('GDP')
plt.ylabel('Life Expectancy')
plt.title('Regressão Linear: GDP vs Life Expectancy')
plt.legend()
plt.show()

print("Embora as diferenças sejam mínimas, a regressão Lasso apresentou o menor erro médio de validação.")
print("Isso pode ser preferido em cenários onde uma ligeira melhoria na validação é desejável.")
print("No entanto, dado o baixo R², é evidente que:O PIB não é suficiente para explicar a variação na expectativa de vida.")
print("É necessário incluir mais variáveis relevantes no modelo para melhorar o desempenho.")
```

Importação de Bibliotecas

Função para Validação Cruzada

Função para Métricas de Desempenho

Leitura e Limpeza dos Dados

Separação dos Dados

Treinamento e Avaliação dos Modelos

Visualização do Ajuste Linear

6. Conclusões

Os resultados esperados foram satisfeitos?

Não todos

7. Próximos passos

Depois desse relatório, quais os próximos passos você sugere para este projeto?